

LAPORAN PRAKTIKUM STRUKTUR DATA

PEKAN 5



Disusun Oleh:

Muhammad Althaf Mulya

Dosen Pengampu:

Wahyudi, Dr. S.T., M.T.

**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS**

2025

Daftar Isi

DAFTAR ISI	2
KATA PENGANTAR	4
1.1 Latar Belakang	5
2.1 Class NodeSLL	6
2.2 Program TambahSLL	7
2.3 Program PencarianDLL	8
2.4 Program HapusDLL	9
2.5 Tugas RumahSakitDanPasien	10
3.1 Ringkasan Hasil Praktikum	18

Kata Pengantar

Puji syukur kehadiran Allah SWT atas limpahan rahmat dan karunia-Nya sehingga laporan praktikum dengan judul “Single Linked List” ini dapat disusun dan diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pelaksanaan kegiatan praktikum pada mata Struktur Data.

Ucapan terima kasih disampaikan kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu, serta kepada asisten laboratorium yang telah memberikan bimbingan selama kegiatan praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi penulisan maupun isi. Oleh karena itu, saran dan kritik yang bersifat membangun sangat diharapkan demi perbaikan di masa yang akan datang. Harapannya, laporan ini dapat memberikan manfaat dan menjadi referensi bagi pembelajaran konsep SLL dalam bahasa pemrograman Java.

Pembahasan

1. Program Node SSL

Tujuan Program

Adalah sebuah kelas yang merepresentasikan satu buah simpul atau node dalam Singly Linked List (SLL) yang terdiri atas 2 properti yaitu int data_3018 sebagai variabel menyimpan nilai dan next_3018 variabel pointer untuk menyimpan alamat node selanjutnya.

```
package pekan5_2511533018;

public class NodeSSL_2511533018
{
    // -- Node bagian data --
    public int data_3018;

    // Pointer ke node berikutnya --
    public NodeSSL_2511533018 next_3018;

    // -- Constructor
    public NodeSSL_2511533018(int data)
    {
        this.data_3018 = data;
        this.next_3018 = null;
    }
}
```

2. Tambah SSL

Tujuan Program

Adalah sebuah kelas yang berisi implementasi bagaimana cara untuk menambahkan sebuah node SSL serta bagaimana cara debugging dan memodifikasi nya.

Terdiri atas 5 method yaitu insert at front yang berguna membuat node baru dan mengarahkan pointer dari node sebelumnya ke sini, dimana variabel head akan menjadi penandanya.

```

public static NodeSSL_2511533018 insertAtFront_2511533018(
    NodeSSL_2511533018 head,
    int value)
{
    NodeSSL_2511533018 new_node_3018 = new NodeSSL_2511533018(value);
    new_node_3018.next_3018 = head;
    return new_node_3018;
}

```

Kemudian method insertAtEnd yang berguna untuk melakukan traversal atau penelusuran menemukan node terakhir lalu menyambungkan node baru di sana.

```

// -- Fungsi menambahkan node di akhir SLL
public static NodeSSL_2511533018 insertAtEnd_2511533018(NodeSSL_2511533018 head, int value)
{
    // Buat sebuah node dengan nilai
    NodeSSL_2511533018 new_node_3018 = new NodeSSL_2511533018(value);

    // Jika list kosong maka node jadi head
    if (head == null)
        return new_node_3018;

    // Simpan head ke variabel sementara
    NodeSSL_2511533018 last_3018 = head;

    // Telusuri ke node akhir
    while (last_3018.next_3018 != null)
    {
        last_3018 = last_3018.next_3018;
    }

    // Ubah pointer
    last_3018.next_3018 = new_node_3018;
    return head;
}

```

Lalu ada method getNode yang berguna untuk mengalokasikan memori dan membuat objek node baru dari data yang diberikan.

```

public static NodeSSL_2511533018 getNode_2511533018(int data)
{
    return new NodeSSL_2511533018(data);
}

```

Setelah itu ada method bernama insertPos yang berfungsi untuk mencari posisi yang diinginkan menggunakan perulangan

```

public static NodeSSL_2511533018 insertPos_2511533018(
    NodeSSL_2511533018 headNode,
    int position,
    int value)
{
    NodeSSL_2511533018 head_3018 = headNode;

    if (position < 1)
        System.out.println("Invalid Position");

    if (position == 1)
    {
        NodeSSL_2511533018 new_node_3018 = new NodeSSL_2511533018(value);
        new_node_3018.next_3018 = head_3018;
        return new_node_3018;
    }
    else
    {
        while (position-- != 0)
        {
            if (position == 1)
            {
                NodeSSL_2511533018 new_node_3018 = getNode_2511533018(value);
                new_node_3018.next_3018 = headNode.next_3018;
                headNode.next_3018 = new_node_3018;
                break;
            }
        }
        headNode = headNode.next_3018;
    }

    if (position != 1)
        System.out.println("Posisi di luar jangkauan.");

    return head_3018;
}

```

Terakhir kita punya method main sebagai entry point dan method printList untuk menampilkan output

```

-
public static void printList_2511533018(NodeSSL_2511533018 head)
{
    NodeSSL_2511533018 curr_3018 = head;

    while (curr_3018.next_3018 != null)
    {
        System.out.print(curr_3018.data_3018 + "-->");
        curr_3018 = curr_3018.next_3018;
    }

    if (curr_3018.next_3018 == null)
    {
        System.out.print(curr_3018.data_3018);
        System.out.println();
    }
}

public static void main(String[] args)
{
    // buat linked list 2 -> 3 -> 5 -> 6
    NodeSSL_2511533018 head_3018 = new NodeSSL_2511533018(2);

    head_3018.next_3018 = new NodeSSL_2511533018(3);

    head_3018.next_3018.next_3018 = new NodeSSL_2511533018(5);

    // cetak list asli
    System.out.println("Tambah 1 simpul di depan: ");

    int data_3018 = 1;

    printList_2511533018(head_3018);

    // tambahkan node baru di belakang
    System.out.println("tambah 1 simpul di belakang: ");

    int data2_3018 = 7;
    head_3018 = insertAtEnd_2511533018(head_3018, data2_3018);

    // cetak update list
    printList_2511533018(head_3018);

    System.out.println("tambah 1 simpul ke data 4: ");

    int data3_3018 = 4;
    int pos_3018 = 4;

    head_3018 = insertPos_2511533018(head_3018, pos_3018, data3_3018);

    // cetak update list
    printList_2511533018(head_3018);
}

```

3. Program Pencarian SSL

Tujuan Program

Sebuah program yang dibuat untuk mengimplementasikan dua operasi dasar dalam membaca data pada Singly Linked List yaitu Traversal (penelusuran seluruh isi list) dan Searching (pencarian data spesifik).

Terdiri atas 3 method yaitu searchKey yang berguna sebagai method yang menggunakan algoritma Linear Search dengan memeriksa setiap node sampai menemukan key dan kembalikan nilai true jika ditemukan dan null jika tidak ditemukan

```
public static boolean searchKey_2511533018(NodeSSL_2511533018 head, int key)
{
    NodeSSL_2511533018 curr_3018 = head;

    while (curr_3018 != null)
    {
        if (curr_3018.data_3018 == key)
            return true;
        curr_3018 = curr_3018.next_3018;
    }
    return false;
}
```

Lalu untuk 2 method lainnya yaitu main method sebagai entry point dan method traversal yang mengunjungi setiap node satu per satu mulai dari head hingga node terakhir. Di setiap perhentian, program mencetak nilai data_3018 ke layar.


```

public static void traversal_2511533018(NodeSSL_2511533018 head)
{
    // mulai dari head
    NodeSSL_2511533018 curr_3018 = head;

    // telusuri sampai pointer null
    while (curr_3018 != null)
    {
        System.out.print(" " + curr_3018.data_3018);
        curr_3018 = curr_3018.next_3018;
        System.out.println();
    }
}

public static void main(String[] args)
{
    NodeSSL_2511533018 head_3018 = new NodeSSL_2511533018(14);
    head_3018.next_3018 = new NodeSSL_2511533018(21);
    head_3018.next_3018.next_3018 = new NodeSSL_2511533018(13);
    head_3018.next_3018.next_3018.next_3018 = new NodeSSL_2511533018(30);
    head_3018.next_3018.next_3018.next_3018.next_3018 = new NodeSSL_2511533018(10);
    System.out.println("Penelusuran SLL: ");

    traversal_2511533018(head_3018);

    int key_3018 = 30;
    System.out.print("cari data " + key_3018 + " = ");
    if (searchKey_2511533018(head_3018, key_3018))
        System.out.print("ketemu");
    else
        System.out.print("tidak ada");
}

```

Writable

Smart Insert

15 : 9 : 317

⋮

4. Program Hapus SSL

Tujuan Program

Adalah sebuah program berisi kelas untuk operasi penghapusan SSL, fokus utamanya adalah bagaimana memanipulasi referensi dari var next_3018 agar node lenyap dari list.

Terdiri atas beberapa method yaitu deleteHead tu yang melakukan pemindahan (penghapusan) pada head suatu SSL pada SSL head lain.

```

public static NodeSSL_2511533018 deleteHead_2511533018(NodeSSL_2511533018 head)
{
    // jika SSL kosong
    if (head == null)
        return null;

    // pindahkan head ke node berikutnya
    head = head.next_3018;

    // return head baru
    return head;
}

```

Kemudian ada removeLastNode yaitu sebuah method untuk menghapus elemen terakhir, program akan mencari node kedua terakhir menggunakan loop dan mengubah pointer enxt nya jadi null.

```

// fungsi menghapus node terakhir SSL
public static NodeSSL_2511533018 removeLastNode_2511533018(NodeSSL_2511533018 head)
{
    // jika list kosong, return null
    if (head == null)
        return null;

    if (head.next_3018 == null)
        return null;

    // temukan node terakhir ke dua
    NodeSSL_2511533018 secondLast_3018 = head;
    while (secondLast_3018.next_3018.next_3018 != null) {
        secondLast_3018 = secondLast_3018.next_3018;
    }

    // hapus node terakhir
    secondLast_3018.next_3018 = null;
    return head;
}

```

Lalu ada method deleteNode untuk menghapus node di posisi tertentu. Program menghubungkan node sebelum target langsung ke node setelah target, sehingga target terlompati dan terhapus.

```

// fungsi menghapus node di posisi tertentu
public static NodeSSL_2511533018 deleteNode_2511533018(NodeSSL_2511533018 head, int position)
{
    NodeSSL_2511533018 temp_3018 = head;
    NodeSSL_2511533018 prev_3018 = null;

    // jika linked list null
    if (temp_3018 == null)
        return head;

    if (position == 1)
    {
        head = temp_3018.next_3018;
        return head;
    }

    // kasus 2: menghapus node di tengah
    // telusuri ke node yang dihapus
    for (int i = 1; temp_3018 != null && i < position; i++)
    {
        prev_3018 = temp_3018;
        temp_3018 = temp_3018.next_3018;
    }

    // jika ditemukan, hapus node
    if (temp_3018 != null)
    {
        prev_3018.next_3018 = temp_3018.next_3018;
    }
    else
    {
        System.out.println("Data tidak ada.");
    }

    return head;
}

```

Kemudian 2 method terakhir adalah entry point (method main) dan method printList untuk menampilkan output.

```

public static void printList_2511533018(NodeSSL_2511533018 head)
{
    NodeSSL_2511533018 curr_3018 = head;

    while (curr_3018.next_3018 != null)
    {
        System.out.print(curr_3018.data_3018 + "-->");
        curr_3018 = curr_3018.next_3018;
    }

    if (curr_3018.next_3018 == null)
    {
        System.out.print(curr_3018.data_3018);
    }
    System.out.println();
}

public static void main(String[] args)
{
    NodeSSL_2511533018 head_3018 = new NodeSSL_2511533018(1);
    head_3018.next_3018 = new NodeSSL_2511533018(2);
    head_3018.next_3018.next_3018 = new NodeSSL_2511533018(3);
    head_3018.next_3018.next_3018.next_3018 = new NodeSSL_2511533018(4);
    head_3018.next_3018.next_3018.next_3018.next_3018 = new NodeSSL_2511533018(5);
    head_3018.next_3018.next_3018.next_3018.next_3018.next_3018 = new NodeSSL_2511533018(5);
    head_3018.next_3018.next_3018.next_3018.next_3018.next_3018.next_3018 = new NodeSSL_2511533018(6);

    // cetak list awal
    System.out.println("List awal: ");
    printList_2511533018(head_3018);

    // hapus head
    head_3018 = deleteHead_2511533018(head_3018);
    System.out.println("List setelah head dihapus: ");
    printList_2511533018(head_3018);

    // hapus node terakhir
    head_3018 = removeLastNode_2511533018(head_3018);
    System.out.println("List setelah simpul terakhir dihapus: ");
    printList_2511533018(head_3018);

    int p = 2;
    head_3018 = deleteNode_2511533018(head_3018, p);
    System.out.println("List setelah posisi 2 blknng dihapus: ");
    printList_2511533018(head_3018);
}
}

```

Tugas Implementasi SLL dengan Rumah Sakit dan Pasien

```
package pekan5_2511533018;

public class Pasien_2511533018
{
    private String namaPasien_3018;
    private String penyakit_3018;
    private int nomorAntrian_3018;
    public Pasien_2511533018 next_3018;

    public Pasien_2511533018(String nama, String keluhan, int nomor)
    {
        this.namaPasien_3018 = nama;
        this.penyakit_3018 = keluhan;
        this.nomorAntrian_3018 = nomor;
        this.next_3018 = null;
    }

    public String getNama_3018()
    {
        return namaPasien_3018;
    }

    public String getPenyakit_3018()
    {
        return penyakit_3018;
    }

    public int getNomor_3018()
    {
        return nomorAntrian_3018;
    }
}
```

Dibuat sebuah class Pasien dengan 3 atribut private untuk enkapsulasi, dan 1 public atribut sebagai pointer, lalu ada constructor serta getter setter.

```
package pekan5_2511533018;
import java.util.Scanner;

public class RumahSakit_2511533018
{
    private Pasien_2511533018 head_3018 = null;
    private int counter_3018 = 0;

    public void daftarPasien_2511533018(String nama, String keluhan)
    {
        counter_3018++;
        Pasien_2511533018 baru_3018 = new Pasien_2511533018(nama, keluhan, counter_3018);

        if (head_3018 == null)
        {
            head_3018 = baru_3018;
        } else
        {
            Pasien_2511533018 temp_3018 = head_3018;
            while (temp_3018.next_3018 != null)
            {
                temp_3018 = temp_3018.next_3018;
            }
            temp_3018.next_3018 = baru_3018;
        }

        System.out.println("Pasien berhasil didaftarkan, No Antrian: " + counter_3018);
    }
}
```

Method untuk menambahkan pasien ke dalam database temporary, dengan input nama dan keluhan.

```

public void tampilkanAntrian_2511533018()
{
    if (head_3018 == null)
    {
        System.out.println("Tidak ada antrian.");
        return;
    }
    Pasien_2511533018 curr_3018 = head_3018;
    System.out.println("== Daftar Antrian ==");

    while (curr_3018 != null)
    {
        System.out.println(curr_3018.getNomor_3018() + ". " + curr_3018.getNama_3018() + " (" + curr_3018.getPenyakit_3018() + "
        curr_3018 = curr_3018.next_3018;
    }
}

```

Method untuk menampilkan antrian pasien.

```

public void cariPasien_2511533018(String namaCari)
{
    Pasien_2511533018 curr_3018 = head_3018;
    boolean ketemu_3018 = false;

    while (curr_3018 != null)
    {
        if (curr_3018.getNama_3018().equalsIgnoreCase(namaCari))
        {
            System.out.println("Pasien ditemukan, No Antrian: " + curr_3018.getNomor_3018());
            ketemu_3018 = true;
            break;
        }
        curr_3018 = curr_3018.next_3018;
    }

    if (!ketemu_3018) System.out.println("Data pasien tidak ditemukan.");
}

```

Method untuk mencari pasien dengan kata kunci namaCari.

```

    public static void main(String[] args)
    {
        RumahSakit_2511533018 rs_3018 = new RumahSakit_2511533018();
        Scanner sc_3018 = new Scanner(System.in);
        int pilihan_3018;

        do {
            System.out.println("\n=== Antrian Rumah Sakit NIM: 2511533018 ===");
            System.out.println("1. Daftarkan Pasien (Insert)");
            System.out.println("2. Panggil Pasien (Delete Head)");
            System.out.println("3. Tampilkan Antrian (Display)");
            System.out.println("4. Cari Pasien (Search)");
            System.out.println("5. Cek Status Antrian");
            System.out.println("6. Keluar");
            System.out.print("Pilihan: ");
            pilihan_3018 = sc_3018.nextInt();
            sc_3018.nextLine();

            switch (pilihan_3018) {
                case 1:
                    System.out.print("Masukkan Nama Pasien : ");
                    String nama_3018 = sc_3018.nextLine();
                    System.out.print("Masukkan Keluhan : ");
                    String keluhan_3018 = sc_3018.nextLine();
                    rs_3018.daftarPasien_2511533018(nama_3018, keluhan_3018);
                    break;

                case 2:
                    rs_3018.panggilPasien_2511533018();
                    break;

                case 3:
                    rs_3018.tampilkanAntrian_2511533018();
                    break;

                case 4:
                    System.out.print("Masukkan Nama Pasien yang dicari: ");
                    String cari_3018 = sc_3018.nextLine();
                    rs_3018.cariPasien_2511533018(cari_3018);
                    break;

                case 5:
                    if (rs_3018.head_3018 == null) {
                        System.out.println("Antrian kosong.");
                    } else {
                        int total_3018 = 0;
                        Pasien_2511533018 temp_3018 = rs_3018.head_3018;
                        while(temp_3018 != null) {
                            total_3018++;
                            temp_3018 = temp_3018.next_3018;
                        }
                        System.out.println("Jumlah total pasien: " + total_3018);
                        System.out.println("Pasien terdepan: " + rs_3018.head_3018.getNama_3018());
                    }
                    break;

                case 6:
                    System.out.println("Terima kasih. Program selesai.");
                    break;

                default:
                    System.out.println("Pilihan tidak valid!");
            }
        } while (pilihan_3018 != 6);

        sc_3018.close();
    }
}

```

Method main sebagai entry point yang menerima input dan memilih program mana yang akan dijalankan berdasarkan input.

Output:

=== Antrian Rumah Sakit NIM: 2511533018 ===

1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar

Pilihan: 1

Masukkan Nama Pasien : Althaf

Masukkan Keluhan : Sakit

Pasien berhasil didaftarkan, No Antrian: 1

=== Antrian Rumah Sakit NIM: 2511533018 ===

1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar

Pilihan: 3

=== Daftar Antrian ===

1. Althaf (Sakit)

=== Antrian Rumah Sakit NIM: 2511533018 ===

1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar

Pilihan: 5

Jumlah total pasien: 1

Pasien terdepan: Althaf

=== Antrian Rumah Sakit NIM: 2511533018 ===

1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar

Pilihan: 4

Masukkan Nama Pasien yang dicari: Althaf

Pasien ditemukan, No Antrian: 1

Pasien terhapus. Antrian

=== Antrian Rumah Sakit NIM: 2511533018 ===

1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar

Pilihan: 2

Memanggil Pasien: Althaf [No: 1]

3.1 Ringkasan

Jadi materi kali ini fokus kepada bagaimana Single Linked List serta implementasi detailnya, mulai dari membuat, menghapus, menelusuri sampai struktur data dasarnya berupa ADT nya.